Université Hassan I
Faculté des Sciences et Techniques de Settat
Département Physique Appliquée – Laboratoire LAVETE

Série de TD/TP 4 – Durée : 2 heures

Notions préalables à la programmation asynchrone

Synchrone/asynchrone • Fonctions comme valeurs • Callbacks et événements

Enseignant : Pr. Abdelfattah Abassi **Grade :** Maître de Conférences
Filière : Master Sciences et Techniques **Année :** 2026–2027

Objectifs et organisation

Cette série couvre les trois premières sections du chapitre « Notions préalables à la programmation asynchrone » : *synchrone et asynchrone*, *fonctions comme valeurs*, *callbacks et événements*.

Partie	Contenu	Durée
TD (papier)	Exercices 1 à 3 : classer, typer, prédire une sortie	40 min
TP (machine)	TP1 à TP5 : chronomètre, fonctions, paramètres nommés, `Timer`, événements	70 min
Rendu	Mise en forme des réponses et dépôt sur GitHub	10 min

Outils. Dart SDK installé (`dart run fichier.dart`) ou, à défaut, DartPad. Attention : `sleep` (`dart:io`) et `Stopwatch` ne fonctionnent pas dans DartPad ; utilisez le SDK pour le TP1.

Consigne de rendu. Toutes les réponses (TD et TP) sont à saisir dans le fichier Markdown `reponses_serie1.md` fourni avec cette série, puis déposées dans votre dépôt GitHub (voir la dernière section). Les questions portent les **mêmes numéros** dans ce sujet et dans le fichier Markdown.

Partie 1 – TD (40 minutes)

Exercice 1. Synchrone, asynchrone, bloquant

≈ 10 min

Q1.1 Classez chaque opération en *rapide* (quasi instantanée) ou *lente* (peut faire attendre l'utilisateur) : addition de deux entiers ; lecture d'un fichier local ; appel à une API sur Internet ; mesure d'un capteur Bluetooth ; saisie d'un texte par l'utilisateur ; tri d'une liste de 10 nombres.

Q1.2 Dites si chaque situation est **synchrone** ou **asynchrone**, en justifiant en une phrase :

(a) rester debout devant la machine à laver jusqu'à la fin du cycle ;
(b) lancer la machine, ranger la cuisine, revenir quand elle sonne ;
(c) attendre devant un guichet que la personne devant vous soit servie ;
(d) envoyer un message puis continuer à travailler en attendant la réponse.

Q1.3 Un écran affiche 60 images par seconde. Combien de temps dure une image (en ms) ? Combien d'images sont perdues pendant une attente de **200 ms** ?

Q1.4 En une phrase chacune, définissez : *bloquant / non bloquant*, puis *concurrence / parallélisme*.

Exercice 2. Les fonctions sont des valeurs

≈ 15 min

Q2.1 Donnez le **type fonction** (de la forme `T Function(U)`) des trois expressions suivantes :

```dart
(String s) => s.length
() => print('ok')
(int a, int b) => a > b
```

Q2.2 Réécrivez en **syntaxe flèche** la fonction ci-dessous. Peut-on faire de même avec une fonction contenant deux instructions ? Pourquoi ?

```dart
int triple(int x) {
  return x * 3;
}
```

Q2.3 Quelle est la sortie exacte du programme suivant ?

```dart
void appliquer(List<int> l, int Function(int) f) {
  for (final x in l) {
    print(f(x));
  }
}

void main() {
  appliquer([1, 2, 3], (x) => x * x);
  appliquer([1, 2, 3], (x) => x + 10);
}
```

Q2.4 Quelle différence y a-t-il entre `f` et `f()` lorsque `f` est une fonction ? Que représente chacune des deux écritures ?

Q2.5 Dans l'appel `connecter('exemple.com', port: 443)`, lequel des deux paramètres est **nommé** ? Donnez la signature d'une fonction `connecter` où `port` vaut 80 par défaut et où `securise` est obligatoire.

Exercice 3. Callbacks et événements

≈ 15 min

Q3.1 Quelle est la sortie du programme suivant (dans l'ordre d'affichage) ? Justifiez.

```dart
import 'dart:async';
```

```dart
void main() {
  print('A');
  Timer(Duration(seconds: 2), () => print('B'));
  Timer(Duration(seconds: 1), () => print('C'));
  print('D');
}
```

Q3.2 Avec la classe `Bouton` ci-dessous, quelle est la sortie ? Identifiez l'**événement**, les **écouteurs** et l'**abonnement**.

```dart
class Bouton {
  final List<void Function()> _ecouteurs = [];
  void abonner(void Function() action) => _ecouteurs.add(action);
  void cliquer() {
    for (final a in _ecouteurs) {
      a();
    }
  }
}

void main() {
  final b = Bouton();
  b.abonner(() => print('X'));
  b.abonner(() => print('Y'));
  b.cliquer();
  b.cliquer();
}
```

Q3.3 Expliquez en trois ou quatre phrases pourquoi les **callbacks imbriqués** (« pyramide d'accolades ») sont difficiles à lire et à maintenir, et ce que `await` permet d'améliorer.

Partie 2 – TP (70 minutes)

Créez un dossier `serie1-asynchrone/` et un fichier `.dart` par TP (`tp1.dart`, `tp2.dart`, ...). Pour chaque TP, copiez votre **code** et la **sortie obtenue** dans `reponses_serie1.md`.

TP1. Mesurer l'attente

≈ 10 min

T1.1 Écrivez un programme qui affiche trois messages séparés par une pause de 1 seconde (`sleep` avec une `Duration` d'une seconde) et mesure la durée totale avec `Stopwatch`. Affichez-la en millisecondes.

T1.2 Quelle durée obtenez-vous ? Est-elle exactement 3000 ms ? Pourquoi ?

T1.3 Pendant les pauses, le programme peut-il faire autre chose ? Quel mot du cours décrit ce comportement ?

TP2. Fonctions anonymes et fonctions d'ordre supérieur

≈ 15 min

T2.1 Écrivez, en syntaxe flèche, une fonction anonyme qui renvoie le **triple** d'un entier ; rangez-la dans une variable de type `int Function(int)` et appelez-la.

T2.2 Écrivez `appliquer(List<int> l, int Function(int) f)` qui affiche `f(x)` pour chaque élément. Testez avec le **carré**, puis avec le **double**, sans modifier `appliquer`.

T2.3 Écrivez `repeter(n, action)`, où `action` est de type `void Function(int)`, qui appelle `action(i)` pour $i = 1, \ldots, n$. Utilisez-la pour afficher « Tour 1 », « Tour 2 », « Tour 3 ».

T2.4 *Défi.* Écrivez `int Function(int) multiplierPar(int k)` qui **renvoie** une fonction multipliant par `k`. Créez `par5` et affichez `par5(7)`.

TP3. Paramètres nommés

≈ 10 min

T3.1 Écrivez `saluer(String nom, {String langue = 'fr'})` qui affiche « Bonjour nom » si `langue` vaut `'fr'`, et « Hello nom » sinon. Testez les deux appels.

T3.2 Écrivez `void presenter({required String nom, int age = 18})`. Appelez-la dans deux ordres différents des arguments, puis **sans** `nom` : que dit le compilateur ?

TP4. Callbacks avec `Timer`

≈ 15 min

T4.1 À l'aide de `Timer`, affichez « Bonjour » après 2 secondes, et le message « Je n'attends pas » **immédiatement** après l'appel du `Timer`. Dans quel ordre s'affichent-ils ?

T4.2 Écrivez `telecharger(nom, quandTermine)`, où `quandTermine` est de type `void Function(String)`, qui appelle `quandTermine` après 1 seconde avec le texte « nom : terminé ». Appelez-la avec un callback qui affiche le message.

T4.3 **Pyramide.** Enchaînez trois téléchargements (`a.txt`, `b.txt`, `c.txt`), chacun ne démarrant qu'une fois le précédent terminé, avec des callbacks imbriqués. Combien de temps dure l'ensemble ? Quel défaut de lisibilité constatez-vous ?

TP5. Événements et écouteurs

≈ 20 min

T5.1 Recopiez la classe `Bouton` de l'exercice 3 et ajoutez une méthode `desabonner` (même type de paramètre que `abonner`) qui retire un écouteur. Montrez qu'après désabonnement, l'écouteur n'est plus appelé. *Indice :* rangez d'abord la fonction dans une variable pour pouvoir la retirer.

T5.2 Créez une classe `Thermometre` qui prévient ses écouteurs (`void Function(double)`) à chaque nouvelle température passée à une méthode `mesurer(double t)`.

T5.3 Abonnez deux écouteurs au thermomètre : l'un affiche la température, l'autre affiche « Alerte chaleur ! » si elle dépasse 30 řC. Simulez quatre mesures.

T5.4 *Pour aller plus loin.* Faites émettre une mesure toutes les secondes avec `Timer.periodic`, puis arrêtez-la après 4 mesures avec `timer.cancel()`. Quel lien voyez-vous avec la notion de `Stream` annoncée dans le cours ?

Rendu : dépôt sur GitHub (10 minutes)

1. Complétez `reponses_serie1.md` (nom, groupe, réponses du TD, code et sorties du TP).
2. Dans un dépôt GitHub personnel, placez le tout dans le dossier `serie1-asynchrone/` :